

缓冲区溢出攻击

版权所有 © 2017 杜文亮,保留所有权利。

允许个人使用。如需在课堂上使用这些习题,仅当该课程采用作者的著作作为教材时方可允许。所有其他用途均须事先征得作者的同意。

S4.1. 下列变量 a 和 x 的地址是如何确定的?也就是说,在运行时,程序是如何知道这两个变量的地址的?

```
void foo(int a) {  
  
    int x;  
}
```

S4.2. 下列代码中的变量位于哪些内存段中?

```
int i = 0;  
void func(char *str)  
{  
    char *ptr = malloc(sizeof(int)); char buf[1024]; int j; static  
    int y;  
  
}
```

S4.3. 请绘制下列 C 函数的函数堆栈帧。

```
int bof(char *str) {  
  
    char buffer[24];  
    strcpy(buffer, str); 返回 1;  
  
}
```

S4.4. 一位学生提议改变栈的增长方式。他建议栈不再从高地址向低地址增长,而是从低地址向高地址增长。这样,缓冲区将被分配在返回地址之上,因此缓冲区溢出不会影响返回地址。请对该提议进行评论。

S4.5. 在书中给出的缓冲区溢出示例(即存在漏洞的 stack.c 程序)中,缓冲区溢出发生在 strcpy() 函数内部,因此跳转到恶意代码是在 strcpy() 返回时,而不是在 foo() 返回时。这种说法正确还是错误?请解释。

S4.6. 缓冲区溢出示例已按如下方式修复。这样安全吗?

```
int bof(char *str, int size) {  
  
    char *buffer = (char *) malloc(size);  
  
    /* 以下语句存在缓冲区溢出问题 */
```

```
strcpy(buffer, str);

    返回 1;
}
```

S4.7. 几位学生在缓冲区溢出攻击中遇到了问题。他们的恶意文件（badfile）构造正确，shellcode 放在了恶意文件的末尾。然而，当他们尝试不同的返回地址时，出现了以下现象。你能解释一下为什么有些地址有效，而有些无效吗？

```
缓冲区地址:0xbffff180
情况 1:retAddr = 0xbffff250 -> 可以获取 shell 访问权限 情况 2:retAddr = 0xbffff280 -> 可以获取 shell 访问权限 情
况 3:retAddr = 0xbffff300 -> 无法获取 shell 访问权限 情况 4:retAddr = 0xbffff310 -> 可以获取 shell 访问权限 情况
5:retAddr = 0xbffff400 -> 无法获取 shell 访问权限
```

S4.8. 以下函数在一个特权程序中被调用。参数 str 指向一个完全由用户提供的字符串（字符串大小不超过 300 字节）。调用此函数时，缓冲区数组的地址为 0xAABB0010，返回地址存储在 0xAABB0050。请写下你将提供给程序的字符串，以便在该字符串被复制到缓冲区且 bof() 函数返回时，特权程序能够执行你的代码。在你的答案中，你不需要写出注入的代码，但字符串中关键元素的偏移量必须正确。注意：这个问题中存在一个陷阱；有些人可能幸运地避开了它，但有些人可能会掉入陷阱。请谨慎。

```
int bof(char *str) {

    char buffer[24];
    strcpy(buffer,str); 返回 1;

}
```

S4.9. ★ ★ ★ 在本

题中，我们将探究堆缓冲区溢出如何导致恶意代码的执行。以下是一段代码执行序列（此处并未展示序列中的所有代码）。在执行此序列的过程中，缓冲区和节点 p 的内存地址（均分配在堆上）如图 1 所示。您可以输入最多 300 字节的数据，这些数据将被复制到缓冲区。您的任务是使缓冲区溢出，这样当目标程序执行到第 行时，它将跳转到您注入到堆内存中的代码（假设堆内存可执行）。返回地址存储在 0xBBFFAACC。

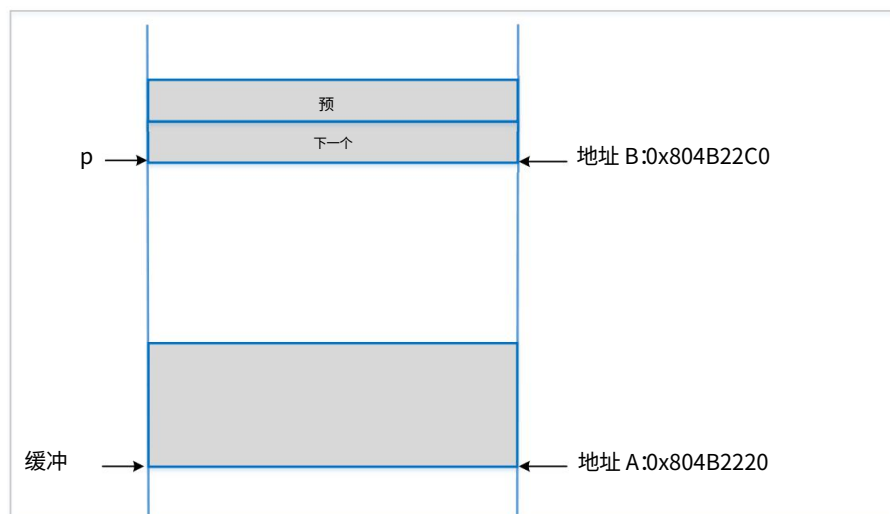


图 1:问题 S4.9 的图示。

```
结构节点
{
    struct Node *next;
    struct Node *pre;
};

// 以下是一段代码执行序列片段。

struct Node *p = malloc(sizeof(struct Node));
struct Node *q;
char *buffer = malloc(100);

/* 省略代码:将节点 p 添加到链表 */

// 以下代码可能存在缓冲区溢出风险
strcpy(buffer, user_input);

// 从链表中移除节点 p
q = p->pre; q->next
= p->next;

返回;
```

提示:您仍然需要将恶意代码的起始地址放入位于 0xBBFFAACC 的返回地址字段中。与栈回滚缓冲区溢出不同,

当然,通过溢出可以到达返回地址字段,现在缓冲区已开启。

返回地址在栈上,但堆上的数据无法通过堆溢出到达栈。你应该利用链表的操作。

列表（行 和 ）用于修改回邮地址字段。

这是一个简化的堆缓冲区溢出漏洞利用方式。链表并非易受攻击程序的一部分；它实际上是操作系统的一部分，用于管理当前进程在堆上的内存。不幸的是，链表也存储在堆上，因此攻击者可以通过溢出应用程序的缓冲区来修改链表中的值。当操作系统操作被篡改的链表时，它可能会改变函数的返回地址，从而触发注入代码的执行。

S4.10. 此题基于 S4.9 题。假设 Node 的结构变为如下形式（在开头添加了一个新的整数字段）。请重新解答 S4.9 题。需要注意的是，在图 1 中，变量 p 现在指向下一个字段下方 4 个字节的区域。

结构节点

```
{  
    int value; struct  
    Node *next; struct Node *pre; };
```

S4.11. 为什么 ASLR 会使缓冲区溢出攻击更加困难？

S4.12. ★

要编写 shellcode，我们需要知道字符串 “/bin/sh” 的地址。如果必须将地址硬编码到代码中，那么在启用 ASLR 的情况下就会变得很困难。

Shellcode 解决了这个问题，而无需在代码中硬编码字符串的地址。

请解释 exploit.c（清单 4.2）中的 shellcode 是如何实现这一点的。

S4.13. 在问题 S4.8 中构建攻击字符串时，在决定返回地址字段的值时有很多选择。你可以使用的最小值是多少？

S4.14. ★

以下函数在远程服务器程序中调用。参数 str 指向一个完全由用户提供的字符串（字符串大小最多为 300 字节）。

缓冲区的大小为 X，我们不知道它的具体值（我们无法调试远程服务器程序）。但是，我们知道缓冲区数组的地址是 0xAABBCC10，并且缓冲区末尾到函数返回地址所在内存区域的距离为 8。虽然我们不知道 X 的确切值，但我们知道它的取值范围在 20 到 100 之间。

请写下你要输入到程序中的字符串，这样当这个字符串被复制到缓冲区，并且 bof() 函数返回时，服务器程序就会运行你的代码。你只有一次机会，所以你需要构造一个字符串，使得即使不知道 X 的确切值也能成功。在你的答案中，你不需要写出注入的代码，但字符串中关键元素的偏移量必须正确。

```
int bof(char *str) {
```

```
char buffer[X];  
strcpy(buffer,str); 返回 1;  
}
```